PHASE

4

Hardware Interrupts: PIC, Timer and Keyboard

Let devices interrupt the CPU, keep time with the PIT, read the keyboard, and give Turk-OS its first interactive prompt

Overall progress toward Turk-OS 1.0 (this phase in red) 22 % → 28 %

Duration	2 weeks
Difficulty	★★★☆☆ (3/5)
You need	Phase 3: GDT, IDT, exception stubs, `register_interrupt_handler`
You will have	A remapped 8259 PIC, IRQ dispatch with end-of-interrupt handling, a 100 Hz timer with uptime and `sleep_ms`, a PS/2 keyboard driver (shift, caps lock, optionally the Turkish Q layout), and a kernel monitor prompt with commands like `help`, `uptime`, `mem` and `reboot`.

1 Why this phase

So far the kernel only does what `kmain` tells it, in order. Real systems are driven by events: a key press, a timer tick, a disk finishing a read. Devices announce these events with **interrupts**, and in this phase you connect the first two devices to the IDT you built in Phase 3.

The timer matters more than it looks. A periodic tick is how the kernel gets control back from a running program, which makes preemptive multitasking possible in Phase 8, and it is also the kernel's clock for timeouts and `sleep`. The keyboard gives you input, and together with the screen that makes Turk-OS interactive for the first time. You will end the phase with a small command monitor inside the kernel. It is not a real shell, since that has to be a user program (Phase 14), but you will use it constantly as a test console from now on.

2 Concepts you need

2.1 The 8259 Programmable Interrupt Controller

Devices don't wire straight into the CPU. On a PC they raise an **IRQ** line on an interrupt controller, the classic 8259A PIC, which forwards one interrupt at a time to the CPU along with a vector number. There are two PICs in a cascade: the master handles IRQ 0–7, and the slave handles IRQ 8–15 and reports through the master's IRQ 2 (Figure 1).

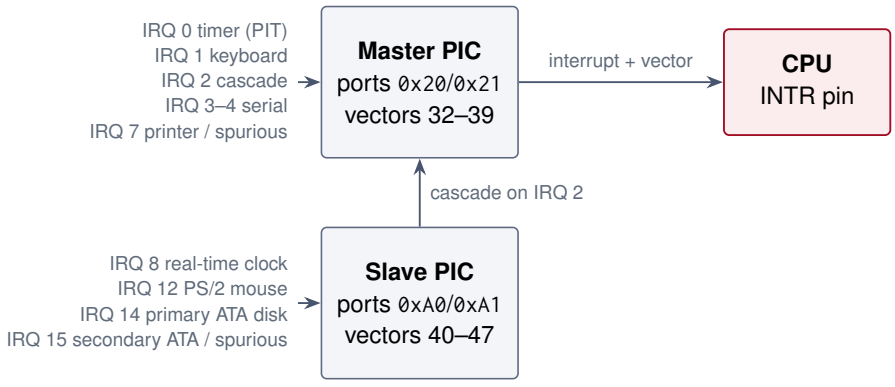


Figure 1: The two cascaded PICs after Turk-OS remaps them. IRQ 14 returns in Phase 12.

Three things about the PIC trip everyone up. First, the BIOS programs the master to use vectors 8–15, which collide with CPU exceptions: the timer's IRQ 0 would arrive as vector 8 and look exactly like a double fault. So the very first job is to **remap** the PICs to vectors 32–47. Second, after handling an IRQ you must send an **end-of-interrupt** (EOI) command, or the PIC never delivers that IRQ again; for IRQs 8–15, both the slave and the master need one. Third, each PIC has a **mask** register where a set bit disables an IRQ line. You will unmask only the devices you have drivers for.

2.2 Timekeeping with the PIT

The 8253/8254 Programmable Interval Timer counts down from a value you choose, driven by a fixed 1 193 182 Hz oscillator, and raises IRQ 0 each time the count reaches zero. For a 100 Hz tick the divisor is $1\,193\,182/100 \approx 11\,932$. Each tick the kernel increments a counter, and everything else follows from it: uptime, `sleep`, timeouts, and in Phase 8 the scheduler's time slices. MOS §5.5 separates *clock hardware* from *clock software* the same way, and also describes soft timers, a useful idea for later.

2.3 The PS/2 keyboard

A keyboard controller (the 8042) sits between the keyboard and the CPU. When you press or release a key it puts a **scan code** in its data port `0x60` and raises IRQ 1. Scan codes identify physical keys, not characters. In the default scan code set 1, pressing a key sends its *make code* and releasing it sends the same code with bit 7 set (the *break code*). Some keys send an `0xE0` prefix first. Turning key events into characters is your job, and it needs a little state: which modifiers (shift, ctrl, alt) are held and whether caps lock is on. The last step is a **layout** table from key to character, and a Turkish Q keyboard needs a different table from a US one. The controller will not report another key until you have read port `0x60`.

2.4 Doing as little as possible in an interrupt

An interrupt handler runs at an unpredictable moment, with interrupts disabled by the interrupt gate, on top of whatever the CPU was doing. It should therefore do the minimum and get out: read the hardware, store the data, send the EOI. The slow work (translating keys, echoing, parsing commands) happens outside, in normal code that takes data from a buffer. MOS §5.3.1 and OSDI §3.2.2 describe this split, and Figure 2 shows it for a key press. The ring buffer between the two halves is your first shared data structure between interrupt context and normal code. Phase 9 comes back to making that safe in general.

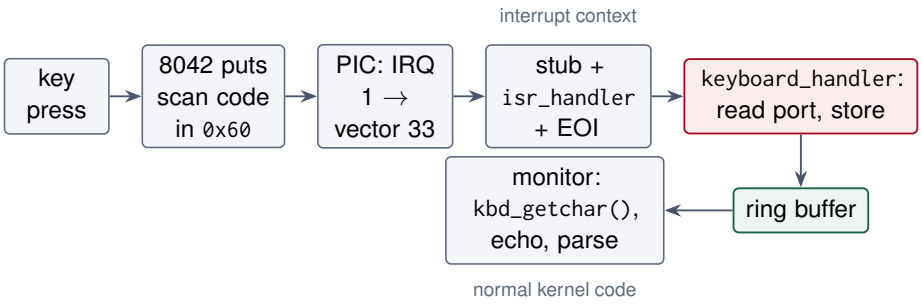


Figure 2: A key press, from the keyboard to the monitor. Only the red box runs with interrupts off.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	LOB	Ch. 6, rest: *Programmable Interrupt Controller (PIC)*, *Reading Input from the Keyboard*	43–45	Tasks 4.1, 4.2 and 4.4.
REQUIRED	LOB	Ch. 14, section *Programmable Interval Timer*	74	Task 4.3.
REQUIRED	MOS	§5.5 *Clocks*: clock hardware, clock software, soft timers	388–394	What a tick is for, and what to build on it.
REQUIRED	OSTEP	Ch. 36 §36.3–36.4 (the canonical protocol; lowering CPU overhead with interrupts)	407–409	Polling versus interrupts, now with real devices.
RECOMMENDED	MOS	§5.2 *Principles of I/O Software*; §5.3.1 *Interrupt Handlers*	351–357	Programmed versus interrupt-driven I/O; what a handler must do.
RECOMMENDED	MOS	§5.6.1 *Input Software* (keyboard software)	394–399	Scan codes, raw versus cooked input, line editing.
RECOMMENDED	OSDI	§2.8 *The Clock Task in MINIX 3*	204–214	A real clock driver: tick handling, alarms, accounting.
RECOMMENDED	OSDI	§3.8.5 *Implementation of the Keyboard Driver*	350–357	Scan code processing and keymaps in MINIX.
OPTIONAL	OSDI	§3.1.4 *Interrupts*; §3.2.2 *Interrupt Handlers*	226–227, 231	Short, clear summary of the hardware side.
OPTIONAL	OSDI	§3.8.1–3.8.2 *Terminal Hardware*, *Terminal Software*	303–316	Canonical mode and special keys, preview of Phase 14.
REQUIRED	LKD	Ch. 7, *Interrupts and Interrupt Handlers*	113–131	Top halves versus bottom halves, interrupt context, interrupt control.
RECOMMENDED	LKD	Ch. 8, *Bottom Halves and Deferring Work*: the introduction and *Work Queues*	133–137, 149–157	The deferred work you add to the IRQ path.
OPTIONAL	LKD	Ch. 11, *Kernel Notion of Time* and *The Ideal HZ Value*	208–211	Why 100 Hz, and what a larger HZ would cost.

Real-system lens: how MINIX 3 does it

MINIX programs the PIC in `kernel/i8259.c (08100)` with the same ICW1–ICW4 sequence as Task 4.1. Its clock interrupt handler in `kernel/clock.c (10400)` does almost nothing: it updates the time, charges the tick to the running process, and only wakes the clock task if something (an expired alarm or time slice) needs attention. That is the “do little in the handler” rule in a real kernel. The keyboard driver in `drivers/tty/keyboard.c (15200)` buffers raw scan codes in its interrupt handler and translates them later through a loadable keymap, which is how MINIX supports many layouts.

Linux, for comparison. LKD Linux splits every interrupt into a **top half**, which acknowledges the device and copies out the urgent data with interrupts masked, and **bottom halves** (softirqs, tasklets, work queues) that do the rest later. Task 4.7 adds the simplest form of this.

4 Build it

Task 4.1 Remap and mask the PIC

Send the four initialisation words to both PICs, moving IRQ 0–7 to vectors 32–39 and IRQ 8–15 to 40–47, then mask everything except the timer and keyboard. Give each write a moment with `io_wait()`; old hardware needed it and it costs nothing.

kernel/arch/i386/pic.c

```
#define PIC1_CMD  0x20
#define PIC1_DATA 0x21
#define PIC2_CMD  0xA0
#define PIC2_DATA 0xA1

void pic_remap(void)
{
    outb(PIC1_CMD,  0x11); io_wait();   /* ICW1: begin initialisation, ICW4 follows */
    outb(PIC2_CMD,  0x11); io_wait();
    outb(PIC1_DATA, 0x20); io_wait();   /* ICW2: master vector offset = 32          */
    outb(PIC2_DATA, 0x28); io_wait();   /* ICW2: slave vector offset  = 40          */
    outb(PIC1_DATA, 0x04); io_wait();   /* ICW3: a slave is attached to IRQ 2       */
    outb(PIC2_DATA, 0x02); io_wait();   /* ICW3: slave's cascade identity is 2      */
    outb(PIC1_DATA, 0x01); io_wait();   /* ICW4: 8086/88 mode                       */
    outb(PIC2_DATA, 0x01); io_wait();
    outb(PIC1_DATA, 0xFC);              /* mask: only IRQ 0 and IRQ 1 enabled       */
    outb(PIC2_DATA, 0xFF);              /* mask every slave IRQ for now             */
}

void pic_send_eoi(uint8_t irq)
{
    if (irq >= 8)
        outb(PIC2_CMD, 0x20);           /* slave first for IRQ 8..15 */
    outb(PIC1_CMD, 0x20);
}
```

Task 4.2 IRQ stubs and dispatch

Add sixteen stubs for vectors 32–47 to `isr.s`. They push a dummy error code and the vector, then reuse `isr_common`, so IRQs get the same `struct regs` as exceptions. Install them in `idt_init` with type `0x8E`. In `isr_handler`, route vectors 32–47 to the registered handler and then send the EOI.

kernel/arch/i386/isr.s (additions)

```
%macro IRQ 2
irq%1:
    push dword 0            ; IRQs have no error code
    push dword %2           ; vector number = 32 + IRQ
    jmp  isr_common
%endmacro

IRQ 0, 32
IRQ 1, 33
; ... one line per IRQ ...
IRQ 15, 47
```

kernel/arch/i386/isr.c (addition at the top of isr_handler)

```
if (r->int_no >= 32 && r->int_no < 48) {
    if (handlers[r->int_no])
        handlers[r->int_no](r);
    pic_send_eoi(r->int_no - 32);
```

```
    return;
}
```

Task 4.3 The timer

Program PIT channel 0 for 100 Hz and count ticks in a handler. Build `timer_ticks()`, `uptime_ms()` and `sleep_ms()` on top. Keep the counter a `volatile uint32_t`: on a 32-bit CPU, a 64-bit counter is read in two halves, and a tick can land between them and give a torn value. At 100 Hz a 32-bit counter lasts about 497 days.

kernel/drivers/pit.c

```
#define PIT_BASE_HZ 1193182

static volatile uint32_t ticks;

static void timer_handler(struct regs *r) { (void)r; ticks++; }

void pit_init(uint32_t hz)
{
    uint16_t divisor = PIT_BASE_HZ / hz;
    outb(0x43, 0x36);                   /* channel 0, low byte then high byte, mode 3 */
    outb(0x40, divisor & 0xFF);
    outb(0x40, divisor >> 8);
    register_interrupt_handler(32, timer_handler);
}

void sleep_ms(uint32_t ms)
{
    uint32_t end = ticks + ms / 10;     /* 10 ms per tick at 100 Hz */
    while ((int32_t)(end - ticks) > 0)  /* this comparison survives wrap-around */
        __asm__ volatile ("hlt");       /* sleep until the next interrupt */
}
```

Enable interrupts only after everything they might touch is ready. The order in `kmain` is: `gdt_init()`, `idt_init()`, `pic_remap()`, `pit_init(100)`, `keyboard_init()`, then `__asm__ volatile ("sti")`.

Verify: Print the tick count once a second for a minute; it goes up by 100 each time. Compare `sleep_ms(5000)` with a stopwatch.

Task 4.4 The keyboard driver

Write `kernel/drivers/keyboard.c`. The handler reads the scan code, updates modifier state on make and break codes, and for ordinary key presses translates through a layout table and puts the character in a ring buffer (your Phase 0 code). Start with the US layout; the table below covers the main block of keys in scan code set 1.

kernel/drivers/keyboard.c

```
static const char keymap_us[128] = {
    0, 27, '1','2','3','4','5','6','7','8','9','0','-','=','\b',   /* 0x00-0x0E */
    '\t','q','w','e','r','t','y','u','i','o','p','[',']','\n',      /* 0x0F-0x1C */
    0,  'a','s','d','f','g','h','j','k','l',';','\'','`',           /* 0x1D-0x29 */
    0,  '\\','z','x','c','v','b','n','m',',','.','/', 0, '*',       /* 0x2A-0x37 */
    0,  ' ',                                                        /* 0x38-0x39 */
};
/* keymap_us_shift[] is the same table with the shifted characters. */

static bool shift, ctrl, caps;
static struct ringbuf kbd_buf;
```

```
static void keyboard_handler(struct regs *r)
{
    (void)r;
    uint8_t sc = inb(0x60);             /* always read, or no further keys arrive */
    bool released = sc & 0x80;
    sc &= 0x7F;

    switch (sc) {
    case 0x2A: case 0x36: shift = !released; return;   /* left / right shift */
    case 0x1D:            ctrl  = !released; return;   /* left ctrl          */
    case 0x3A: if (!released) caps = !caps;  return;   /* caps lock toggles  */
    }
    if (released)
        return;

    char c = keymap_us[sc];
    bool letter = (c >= 'a' && c <= 'z');
    if (shift ^ (caps && letter))
        c = keymap_us_shift[sc];
    if (c)
        rb_put(&kbd_buf, c);
}
```

The consumer side waits for a key with the processor halted. Checking the buffer and then halting has a tiny race: if the key arrives between the check and `hlt`, you sleep until the next tick. On x86 the classic cure is to disable interrupts for the check and use `sti; hlt` together, because `sti` only takes effect after the following instruction, so no interrupt can slip in between them.

kernel/drivers/keyboard.c (consumer)

```
char kbd_getchar(void)
{
    char c;
    for (;;) {
        __asm__ volatile ("cli");
        if (rb_get(&kbd_buf, &c)) {
            __asm__ volatile ("sti");
            return c;
        }
        __asm__ volatile ("sti; hlt");  /* enable interrupts and sleep, atomically */
    }
}
```

Verify: Letters, digits, punctuation, shift and caps lock all produce the right characters. Holding a key produces auto-repeat, because the keyboard itself resends the make code.

Task 4.5 The Turkish Q layout (optional)

Add a second table and a way to switch between them, for example a `layout tr` command. On a Turkish Q keyboard the letter keys sit on the same scan codes as their US neighbours: `0x17` gives ı (and I with shift), `0x1A` ğ, `0x1B` ü, `0x27` ş, `0x28` i (and İ with shift), `0x33` ö and `0x34` ç. Because several of these characters are outside ASCII, let the keyboard produce Unicode code points instead of `char`, and let the console convert them for the screen. Code page 437 can show ç, ö and ü directly; print a placeholder for ğ, ş and ı until Phase 15 gives you a font that has them.

Task 4.6 Line input and the kernel monitor

Write `readline(buf, size)`, which echoes characters, handles backspace (it uses your VGA driver's `\b`) and returns on Enter. Then write the monitor loop: print a prompt, read a line, split it at spaces, and look the first word up in a command table. Every later phase adds commands here.

Command	What it does
`help`	list all commands with one-line descriptions (from the command table itself)
`clear`, `echo <text>`, `color <fg> <bg>`	exercise the VGA driver
`uptime`, `ticks`	show time since boot from the PIT counter
`mem`	memory size from the Multiboot information (a real report arrives in Phase 5)
`crash <kind>`	run one of your Phase 3 exception tests
`dmesg`	print the log ring buffer of Task 2.7, oldest line first
`reboot`	wait until `inb(0x64) & 0x02` is clear, then `outb(0x64, 0xFE)` to pulse the CPU reset line

Verify: A session like `turk> uptime` prints the right time; backspace edits the line; `dmesg` replays the boot messages that scrolled away; `reboot` restarts QEMU and you see the boot banner again.

Task 4.7 Keep handlers short: top and bottom halves

While a handler runs, the PIC holds back every interrupt of equal or lower priority, and with interrupts disabled nothing else runs at all. So split each handler the way Linux does (LKD ch. 7–8): the **top half** does only what cannot wait, such as reading the device register and acknowledging it, and queues the rest as **deferred work**; `irq_dispatch` runs the queued work after the EOI, with interrupts enabled again. Turn the keyboard into the first user of it: the top half stores the raw scan code, the bottom half does the modifier and layout logic. In Phase 12 the disk handler follows the same rule: it only acknowledges the device and wakes the waiting task.

kernel/arch/i386/irq.c (deferred work)

```
struct work { void (*fn)(void *arg); void *arg; };
static struct work queue[64];              /* filled by top halves, drained here */
static volatile unsigned head, tail;

void defer_work(void (*fn)(void *), void *arg)  /* called by a top half, interrupts off */
{
    if (head - tail == 64)
        panic("deferred-work queue full");
    queue[head++ % 64] = (struct work){ fn, arg };
}

void run_deferred_work(void)                    /* end of irq_dispatch, after the EOI */
{
    __asm__ volatile ("sti");                   /* other interrupts may now arrive */
    while (tail != head) {
        __asm__ volatile ("cli");
        struct work w = queue[tail++ % 64];
        __asm__ volatile ("sti");
        w.fn(w.arg);
    }
    __asm__ volatile ("cli");                   /* the stub's iret restores the old state */
}
```

Bottom halves still run in interrupt context: they must not sleep or wait for a lock that a task holds, which is the rule you meet again in Phase 9. Nested interrupts may drain the queue too, so keep each piece of work self-contained.

Verify: The keyboard behaves exactly as before. A deliberately slow bottom half (a busy loop of about 50 ms) no longer makes `uptime` lose ticks, because the timer interrupt still gets in while it runs.

5 Testing and debugging

Run with `-d int -no-reboot` once to watch the interrupts arrive: you should see a stream of `v=20` (timer) entries and a `v=21` for every key event. The QEMU monitor command `info pic` shows the PIC state, including the mask and in-service registers.

Symptom	Likely cause	What to do
Exactly one timer interrupt ever arrives	No EOI sent, so the PIC never delivers IRQ 0 again	Send the EOI after every IRQ, including when no handler is registered.
Double fault (vector 8) as soon as you run `sti`	PIC not remapped: IRQ 0 arrives as vector 8	Remap before `sti`. This is the classic first-IRQ bug.
The keyboard works for one key, then stops	Port `0x60` not read in the handler, or no EOI	Read the scan code unconditionally at the start of the handler.
Timer much too fast or slow	Divisor wrong, or overflowed 16 bits	For 100 Hz it is 11 931; the minimum frequency is about 19 Hz.
Random characters appear or disappear	The handler and the reader race on the ring buffer indices	One producer and one consumer with `volatile` indices is safe here; Phase 9 generalises it.
Occasional interrupt on vector 39 (IRQ 7) or 47	Spurious IRQ from the PIC	Read the in-service register (OCW3 `0x0B`); if the bit is clear, it was spurious: skip the EOI (for IRQ 15, send it to the master only).

6 Milestone

Milestone: Turk-OS is interactive

- ☐ PIC remapped to 32–47; only the timer and keyboard are unmasked; every IRQ gets an EOI.
- ☐ Ticks arrive at 100 Hz; `uptime` is accurate and `sleep_ms` works.
- ☐ Typing echoes correctly with shift, caps lock and backspace.
- ☐ The monitor runs `help`, `clear`, `uptime`, `mem`, `crash` and `reboot`.
- ☐ Left idle for ten minutes at the prompt, the kernel neither crashes nor spins: it halts between interrupts.

Self-check questions

1. Why must the PICs be remapped, and what exactly goes wrong if you skip it?
2. What is an EOI, and why do IRQs 8–15 need two of them?
3. Compute the PIT divisor for 1000 Hz. What would that frequency cost and what would it buy you?
4. How is a key release distinguished from a key press in scan code set 1?
5. Why should an interrupt handler do as little as possible? Where does the rest of the work go?
6. Why is `sti; hlt` safe while `sti` followed later by `hlt` is not?

7 Going further

Read the real-time clock (CMOS ports `0x70`/`0x71`) and add a `date` command that prints the date in Turkish format (`25.09.2026 14:05`). Make the PC speaker beep using PIT channel 2. Add arrow-key history to `readline` (arrows send `0xE0`-prefixed codes). If you want to prepare for multiprocessor support, read about the local APIC and I/O APIC, which replace the PIC on modern machines; you will need them only if you take the SMP track in Phase 15.

Next: **Phase 5 Physical Memory Management** →

Books used in this plan

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. **LOB** and **OSTEP** are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.